Corporación Favorita Sales

Competition: Grocery Store Sales: Time Series Forecasting

1 Introduction

This case study is to review and better understand the dynamics underlying the machine learning competition on Kaggle to predict store sales. This is a starter competition that is forever ongoing and can help one hone skills in forecasting when working with time series datasets. In this document, the data will be reviewed as well as various machine learning models used and more. Part of the intent is to be able to return to this document to use as a referential tool in future projects.

2 The Problem and the Data

This project uses data from a grocery store with branches throughout the South American country Ecuador to predict future sales of its products. The models will allow one to better foresee how much of each product category will be sold per store using historical data tied to information regarding regional holidays, oil prices, and more.

2.1 The problem

2.1.1. Supervised Regression Problem

Since a continuous numerical outcome (sales) is sought after from the predictions and all the data is labeled, this is a supervised regression problem in machine learning. Various regression models will be attempted, including the 'big three' gradient-boosting algorithms (i.e. XGBoost, LightGBM, & CatBoost).

Seeing that a model is needed to predict sales based on multiple variables (multivariate forecasting problem), a tree based model will be better equipped to handle the dimensionality. Instead of building models on data on a specific product family from a specific store, the models below are derived from data from all branches of the store as well as all product families. If it were a univariate analysis, using data just from a specific store and product family, then the linear regression algorithm would be more viable and promising results could be achieved via a SARIMA model.

Though linear regression will be tried to see it in comparison, it is expected for the decision-tree based models to work at handling the categorical data better; adding the self-correcting mechanisms of gradient boosting, the 'big-three' should prove to be the best.

2.2 Variables and Features

2.2.1. Target Variable: 'Sales'

The 'Sales' feature acts as the target variable, it is a continuous variable whose values must be forecasted based on historical data provided.

2.2.2. Contributing Variables

The test set given provides five starting features: 'id' (transaction number), 'date', 'store_nbr' (the branch number for this specific store), 'family' (product category), 'on promotion' (signifying how many products per this family will be on promotion that day). After combining datasets and some feature engineering, 15 variables will be used to train the algorithm:
(*The ones being used for training are bolded below*)

- 'id' serves as the index number for when submitting the data
- 'date' is used to directly create features contributing to the training:
  - **'time_step'**
  - **'dayofweek'**
  - **'is_month_start'**
  - **'is_quarter_end'**
  - **'Is_year_end'**
- **'store_nbr'** is the categorical variable that will help tie together many datasets
- **'family'** is a strongly influential categorical variable for the model training
- **'on_promotion'** a discrete numerical variable that will help for training as well

Followed by:

- **'is_holiday'** variable created using the holiday data set
- **'dcoilwtico'** is the feature showing the oil prices of the dates in the test data
- Rolling Mean (**'roll_mean_7', 'roll_mean_14', 'roll_mean_28')** are three features showing average sales over recent windows
- **'lag_28'** a lag feature that shifts the target variable's values 28 days forward to show if recent-past data from the target variable can give insights into seasonality
- **'DOW_mean_sales'** is a feature aggregated from the average amount of sales per day-of-the-week and product category

2.3 Overview of the 7 datasets

a) Train.csv is the main dataset used to train the machine learning models
   - Consists of 3,000,888 rows and 6 features:
     1. Store_nbr : 54 different branches
     2. Date : From 1/1/13 to 8/15/17
     3. Id : 3,000,888 unique ids for each transaction
     4. Onpromotion: Amount of products of that category on sale on that date
     5. Family: 33 types of product categories being sold
     6. Sales: Target variable;

b) Test.csv will be used to test the models and create the 'csv' file to submit to the competition
   - Consists of 28,512 rows and 4 features (id, date, store_nbr, family)
   - Has a lead time of 15 days (starts on 8/16/17)
   - Forecast horizon until 8/31/17

c) Sample_submission.csv is simply a file showing the desired format for submitting to the competition

d) Stores.csv provides general info for each branch, its location and type of store it is
   - 54 records for each branch of the store (store_nbr)
   - 4 more features showing: city, state, type, cluster (where similar stores are grouped)

e) Oil.csv provides the daily oil price for all the dates within the training and test datasets
   - Per Kaggle: Ecuador is an oil-dependent country and its economic health is highly vulnerable to shocks in oil prices.
   - The only data set with null values (had 43 NaN's for price).

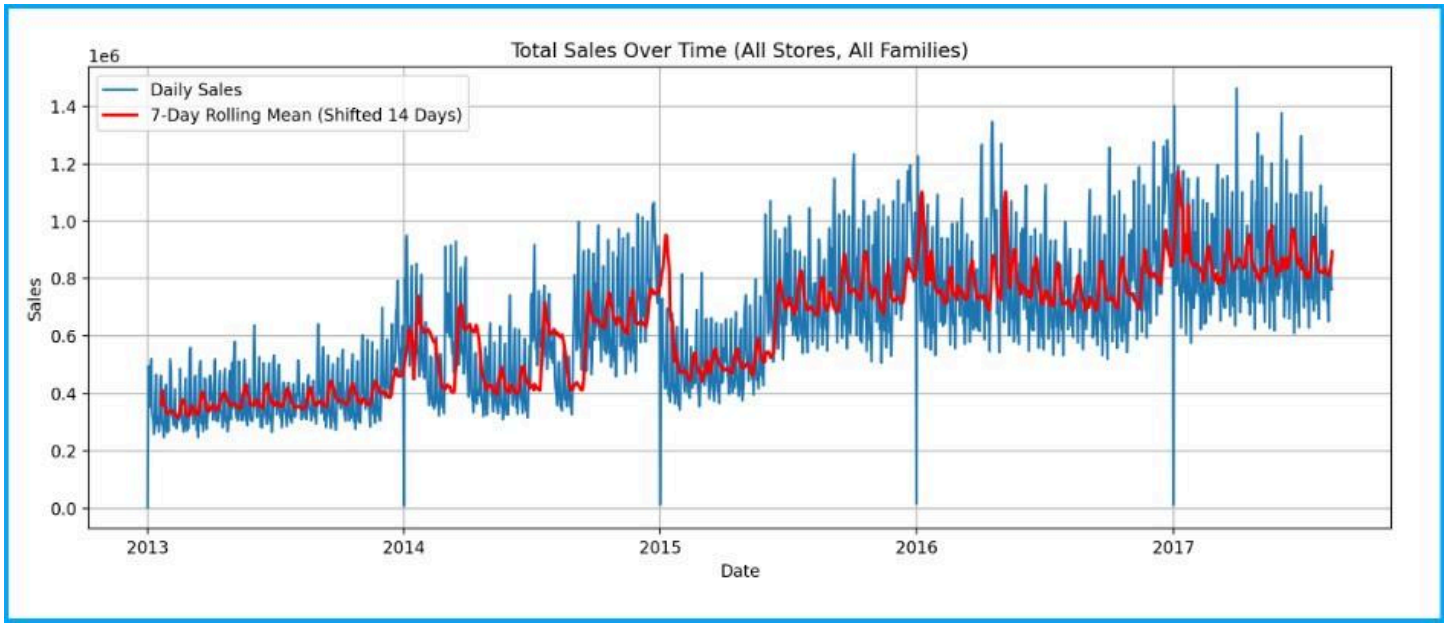
f) Holiday_events.csv has info related to holidays/events within the country
   - 350 records and 6 features (date, type, locale, locale_name, description, transferred)
      - Locale feature indicates if the event is local to the city, region, or is countrywide
      - Locale_name is the name of the city in which the event is based
      - Type and Transferred cover the events that are moved by the government

g) Transactions.csv shows the total amount of transactions per store per date
   - 83,488 records and 3 features (date, store_nbr, transactions)
   - Not used for the competition submission, but has valuable information if needed for EDA

2.4 EDA (Exploratory Data Analysis)

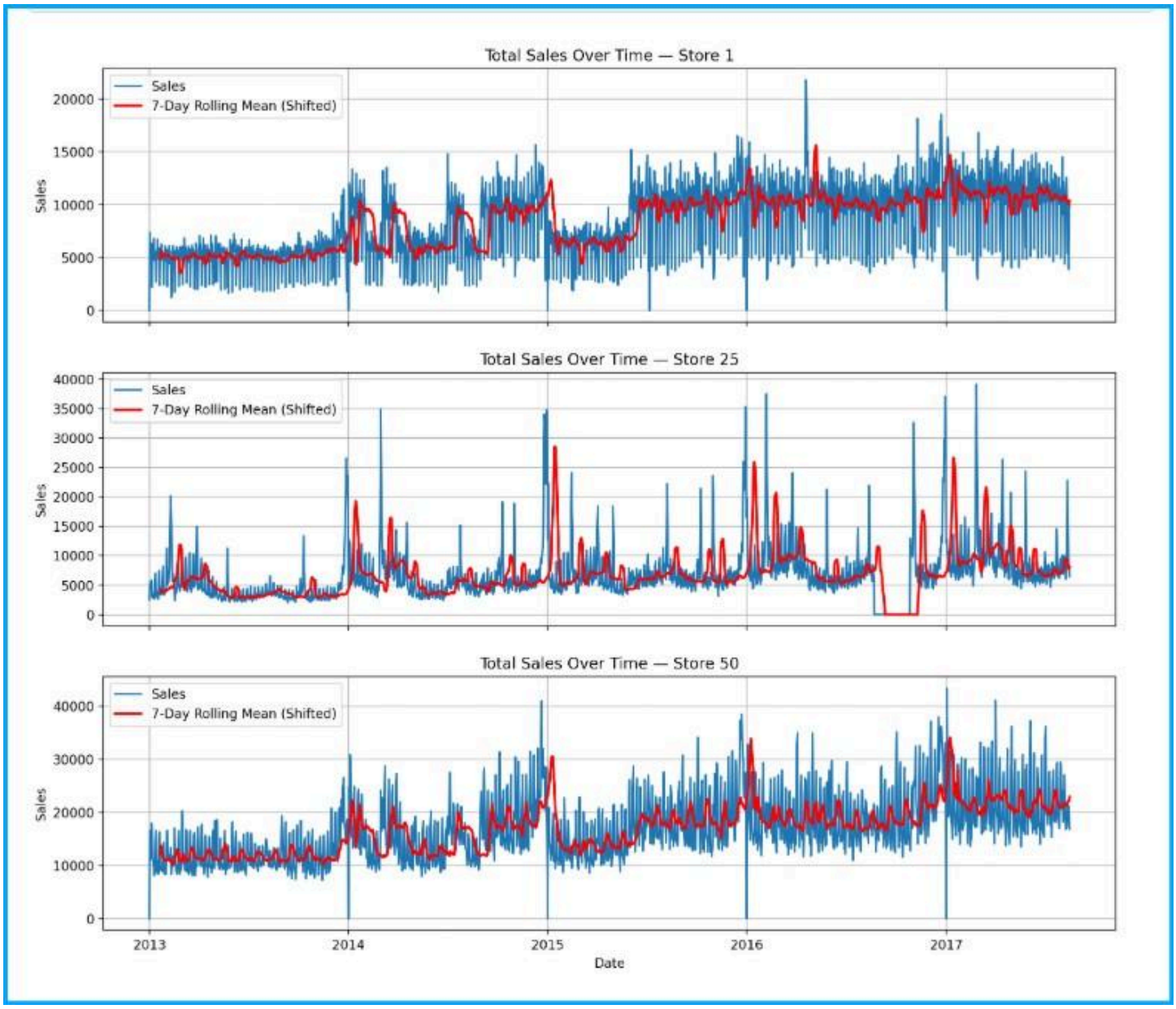
2.4.1 Sales Over Time

In the lineplots below, one is presented with the amount of sales over time. All plots also include a red line delineating the average value of the previous 7 day window and shifted 14 days to prevent data leakage when used as a training feature.



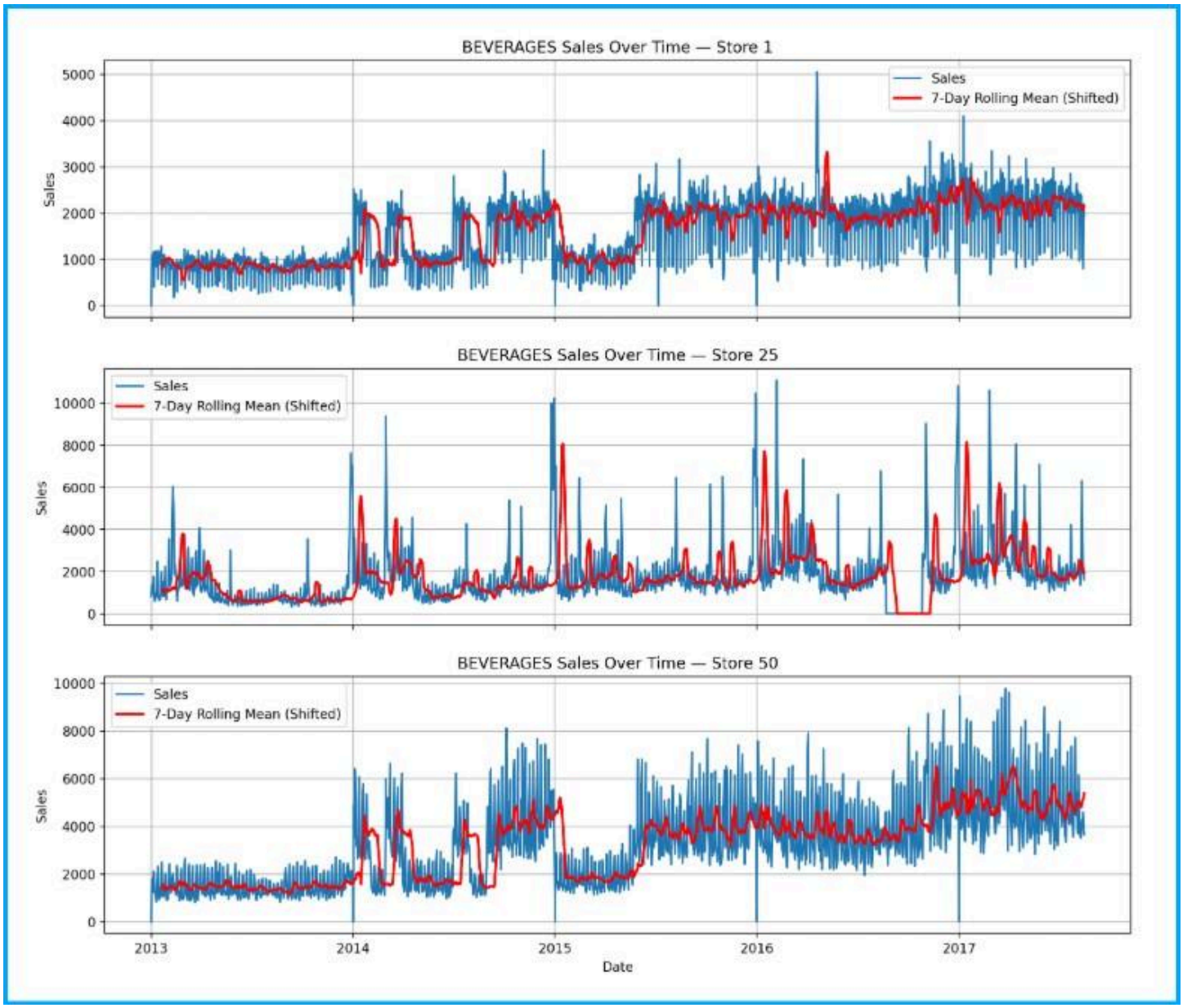
Graph 2.4.1-A - Total Sales over time for all stores and all product families

When looking at the data covering all the stores and product categories, the general picture is a slow and steady growth in total sales; the average amount of daily sales between 2013 and 2014 appears to be hovering around 400,000 units and by 2017 it is reaching around 800,000 units.



Graph 2.4.1-B - Total Sales over time for three specific stores

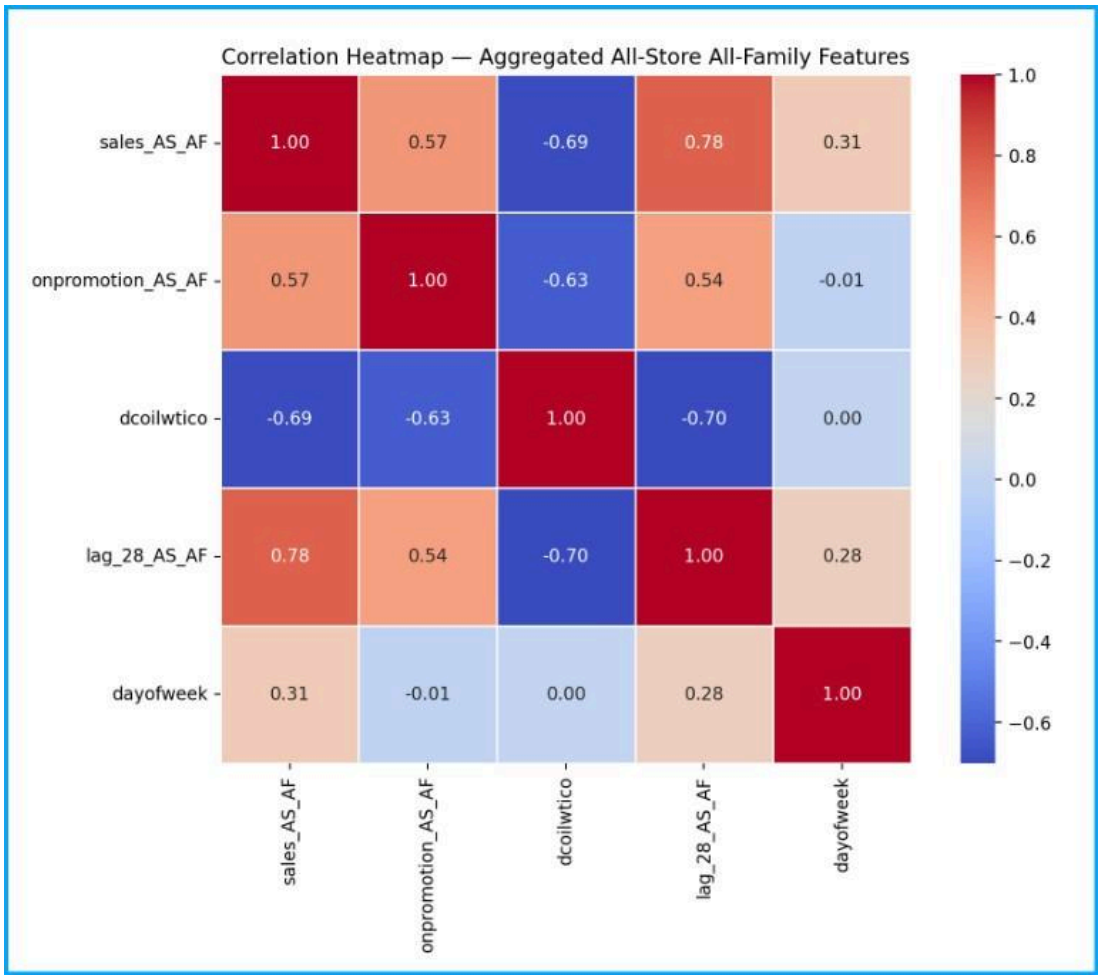
When taking a peak at three specific stores and their daily sales of all product categories, there are similarities to be had; all three stores are dealing at roughly that same scale and show similar trends in their data.



Graph 2.4.1-C - Total Sales of specific family of product at the three stores
The same three stores here are showing the 'Beverage' family of products sales; all three stores show similar trends, though stores 1 and 50 show more similarity.

2.4.2 Collinearity Check of Numeric Features

The heatmap below shows the correlation between five numeric features with one of them being an aggregated form of the target variable; 'Sales' was aggregated to show the amount of sales across 'all stores and all product families' (AS_AF). A similar aggregation was also done with the 28 day lag feature and product promotion feature. Those three features were then compared with the price of oil in the country that day and an integer representing the day of the week.



Graph 2.4.2-A - HeatMap showing correlation of numeric variables (including Target Variable)
As expected, there is some correlation (0.78) between 'sales_AS_AF' and the 'lag_28_AS_AF' feature that was derived from it. The 'dcoilwtico' feature shows a slightly stronger negative correlation with the lag feature (-0.70) than with the main sales feature (-0.69). The feature showing the amount of products on promotion has a weak correlation (0.57) with the sales feature.

3 Setup before model usage

Here the initial setup processes implemented to engineer the data so that it can be used efficiently for the various models are covered.

3.1 Initial imports and dataframe creation

```
import numpy as np, pandas as pd, matplotlib.pyplot as plt, seaborn as sns

sample_df =
      pd.read_csv(""/kaggle/input/competitions/store-sales-time-series-forecasting/sample.csv")
train_df =          pd.read_csv("...train.csv",parse_dates=['date'])
test_df =           pd.read_csv("...test.csv",parse_dates=['date'])
transactions_df =   pd.read_csv("...transactions.csv",parse_dates=['date'])
stores_df =         pd.read_csv("...stores.csv")
holidays_df =       pd.read_csv("...holidays_events.csv",parse_dates=['date'])
oil_df =            pd.read_csv("...oil.csv",parse_dates=['date'])
```

3.2 Checking for Null Values

Initial testing of all the dataframes showed that only the dataset with the oil prices contained any null values.

3.2.1. Nulls in the oil prices dataset

Checking for null values in oil_df showed 43 values for the price feature were missing; this was corrected by doing both 'forward fill' and 'backward fill'. First the 'forward fill' took the last known value before the NaN and used that as the value. The 'backward fill' was also done to ensure any nulls from the beginning of the dataset were filled from the nearest future value.

```
oil_df.isnull().sum()
oil_df['dcoilwtico'] =     oil_df['dcoilwtico'].fillna(method='ffill')
oil_df['dcoilwtico'] =     oil_df['dcoilwtico'].fillna(method='bfill')
```

3.2.2. Nulls after merging holidays dataset to main dataframe

Null values appeared after merging the holidays dataframe to the main training dataframe; any record with a date that did not have an event/holiday, had features containing NaN as its value. This was addressed by creating a single feature 'is_holiday' which acts as a boolean (0 or 1 for possible values), so all null values were simply represented as the 0 for this new feature.

3.3 Merging the datasets

In order for a comprehensive use of the holiday dataset that is sometimes only relevant to a specific city, it was important to first merge the stores_df to the train_df / test_df to add the features of location per the store_nbr. The dataframe containing oil prices were also merged to provide access to this additional feature.

3.3.1. Merging the 'Store info' and 'Oil price' datasets to train/test sets

```
train_data = train_data.merge(stores_df[['store_nbr','city','state']], on='store_nbr', how='left')
test_data  = test_data.merge(stores_df[['store_nbr','city','state']], on='store_nbr', how='left')

train_data = train_data.merge(oil_df[['date','dcoilwtico']], on='date', how='left')
test_data  = test_data.merge(oil_df[['date','dcoilwtico']], on='date', how='left')
```

3.3.2. Creating integer booleans in train/test based on holidays

The holiday dataset splits into national/regional/local. MultiIndexes were built to efficiently check whether each (state, date) or (city, date) pair corresponds to a holiday, which then creates boolean holiday indicators that then constrict into a single 'is_holiday' feature.

```
hol =               holidays_df.copy()
national =          hol[hol['locale'] == 'National'][['date']]
regional =          hol[hol['locale'] == 'Regional'][['date', 'locale_name']]
local    =          hol[hol['locale'] == 'Local'][['date', 'locale_name']]

regional_index =          (regional.set_index(['locale_name', 'date']).index)
local_index =             (local.set_index(['locale_name', 'date']).index)
train_idx_reg =           pd.MultiIndex.from_frame(train_data[['state', 'date']])
train_idx_loc =           pd.MultiIndex.from_frame(train_data[['city', 'date']])
test_idx_reg =            pd.MultiIndex.from_frame(test_data[['state', 'date']])
```

```
test_idx_loc =            pd.MultiIndex.from_frame(test_data[['city', 'date']])

train_data['reg_holi'] =  train_idx_reg.isin(regional_index).astype(int)
test_data['reg_holi']  =  test_idx_reg.isin(regional_index).astype(int)
train_data['loc_holi'] =  train_idx_loc.isin(local_index).astype(int)
test_data['loc_holi']  =  test_idx_loc.isin(local_index).astype(int)

train_data['nat_holi'] =  train_data['date'].isin(national['date']).astype(int)
test_data['nat_holi']  =  test_data['date'].isin(national['date']).astype(int)

train_data['is_holiday'] =       (train_data['nat_holi'] | train_data['reg_holi'] |
                                  train_data['loc_holi']).astype(int)
test_data['is_holiday'] =        (test_data['nat_holi'] | test_data['reg_holi'] |
                                  test_data['loc_holi']).astype(int)
```

3.4 Creating a ‘time step’ feature

The earliest date from the data set was found and subtracted from all other dates to create the time-step feature:

```
train_data['date'].min()
train_data['time_step'] =        (train_data['date'] - pd.Timestamp('2013-01-01')).dt.days
test_data['time_step'] =         (test_data['date'] - pd.Timestamp('2013-01-01')).dt.days
```

3.5 Creating features for each component of the date

The date feature was broken down into separate features (year, month, day, dayofweek, is_weekend, is_month_start, is_quarter_start, is_quarter_end, is_year_start, is_year_end). After testing, using all the features proved superfluous and only the following features from this list were used finally: dayofweek, is_month_start, is_quarter_end, is_year_end.

```
for df in [train_data, test_data]:
    df["date"] =                        pd.to_datetime(df["date"])
    df["year"] =                        df["date"].dt.year
    df["month"] =                       df["date"].dt.month
    df["day"] =                         df["date"].dt.day
    df["dayofweek"] =                   df["date"].dt.dayofweek
    df['is_weekend'] =                  (df['dayofweek'] >= 5).astype(int)
    df.loc[:, 'is_month_start'] =       df['date'].dt.is_month_start.astype(int)
    df.loc[:, 'is_quarter_start'] =     df['date'].dt.is_quarter_start.astype(int)
    df.loc[:, 'is_quarter_end'] =       df['date'].dt.is_quarter_end.astype(int)
    df.loc[:, 'is_year_start'] =        df['date'].dt.is_year_start.astype(int)
    df.loc[:, 'is_year_end'] =          df['date'].dt.is_year_end.astype(int)
```

3.6 Creating ‘rolling mean’ and lag features

In order to avoid leakage when creating these features, the train and test datasets were combined into one dataframe containing the full timeline; additionally a 14 day shift was applied when creating the rolling mean to account for the 14 day lead time of the test set.

```
full =              pd.concat([train_data, test_data], sort=False)
full =              full.sort_values(['store_nbr', 'family', 'date'])
```

```
for window in [7, 14, 28]:
    full[f'roll_mean_{window}'] = (full.groupby(['store_nbr', 'family'])['sales']
        .transform(lambda x: x.shift(14).rolling(window).mean()))
for lag in [1,7,14, 28, 42]:
    full[f'lag_{lag}'] = (full.groupby(['store_nbr', 'family'])['sales']
        .transform(lambda x: x.shift(lag)))

# Split back
train_data =        full[full['sales'].notna()].copy()
test_data  =        full[full['sales'].isna()].copy()
```

3.7 Target Mean Encoding (feature engineering)

Feature engineering was used to create a new feature that is an aggregate of other features. Namely: store number, product family, and day of week had their mean processed and created as a new feature. This would tell per store, how many of this product category sells on each day of the week on average. This was created as a separate dataframe and merged back into the train and test datasets.

```
dow_encoding = (train_encoding_base.groupby(['store_nbr', 'family', 'dayofweek'])['sales']
    .mean().reset_index().rename(columns={'sales': 'DOW_mean_sales'}))

train_data = train_data.merge(dow_encoding,on=['store_nbr', 'family', 'dayofweek'],how='left')
test_data =  test_data.merge(dow_encoding,on=['store_nbr', 'family', 'dayofweek'],how='left')
```

3.8 Categorical feature encoding; X/Y Split

Three different methods were used for encoding the categorical features. Initially only the ‘family’ feature was treated as categorical, later models correctly included ‘store_nbr’ since its numeric values are representative of an ID and hence are qualitative and not quantitative. If fed as a numeric feature, it could cause splits in the decision trees on thresholds like ‘store_nbr < 17’ which would be counter-intuitive.

3.8.1. Get_dummies & OneHotEncoder

These methods are similar in how they encode categories; they essentially create a new column for each possible value of the categorical feature and then give it a value of 1 or 0. It should be noted that OneHotEncoder is the recommended method for a production pipeline and consistent encoding that can safely handle unseen categories.

pd.get_dummies():

```
features =   ["month","day","year","dayofweek","store_nbr","family","onpromotion"]

X =          pd.get_dummies(train_data[features], columns=["family"])
X_test =     pd.get_dummies(test_data[features], columns=["family"])
X, X_test =  X.align(X_test, join='left', axis=1, fill_value=0)
y =          train_data["sales"]
```

OneHotEncoder():

```
from sklearn.preprocessing import OneHotEncoder
```

```
encoder =           OneHotEncoder(drop='first', sparse_output=False,handle_unknown='ignore')
encoded_train =     encoder.fit_transform(train_data[['family']])
encoded_train_df =  pd.DataFrame(encoded_train,columns=encoder.get_feature_names_out(['family']))
train_data =              pd.concat([train_data.drop('family', axis=1), encoded_train_df], axis=1)
encoded_test =      encoder.transform(test_data[['family']])
encoded_test_df =   pd.DataFrame(encoded_test,columns=encoder.get_feature_names_out(['family']))
test_data =               pd.concat([test_data.drop('family', axis=1), encoded_test_df], axis=1)

features =   ['time_step', 'store_nbr'] + family + ['onpromotion']

X =          train_data.loc[:, features]
X_test =     test_data.loc[:, features]
y =          train_data.loc[:, 'sales']
```

3.8.2. Label Encoder

This method takes the categorical feature and assigns an integer for each of the possible values; this may lead to a risk of implying order, but can be used safely when using a tree-based model that does not assume numeric distance.

```
from sklearn.preprocessing import LabelEncoder

features =   ['time_step', 'store_nbr', 'family','onpromotion','dayofweek','is_holiday',
            'roll_mean_7', 'roll_mean_14', 'roll_mean_28','lag_28','dcoilwtico','is_month_start',
            'is_quarter_end','is_year_end','DOW_mean_sales']

X =          train_data.loc[:, features].copy()
X_test =     test_data.loc[:, features].copy()
y =          train_data['sales'].copy()

le =                LabelEncoder()
X[col] =            le.fit_transform(X['family'])
X_test[col] =       le.transform(X_test['family'])
```

3.9 Validation subset for training ‘GB’ models

Each of the three gradient boosting models used required a validation subset for training purposes.

```
val_date =   pd.Timestamp('2017-07-01')
train_mask = train_data['date']<val_date
val_mask =   ~train_mask

X_train, y_train =        X[train_mask], y[train_mask]
X_val, y_val =            X[val_mask], y[val_mask]
```

4 Models & Algorithm Implementation

The following models were used: LinearRegression, RandomForestRegressor, CatBoostRegressor, LightGBM, and XGBoost.

4.1 LinearRegression

A very basic starting model good for forecasting of trends and seasonality on simple linear datasets.

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X, y)
~~~
y_pred =            pd.DataFrame({'sales':model.predict(X_test)})
submission =        pd.concat([test_data['id'], y_pred], axis=1)
submission.to_csv('submission.csv', index=False)
```

BEST SCORE: 2.37294 (V2)

4.2 RandomForestRegressor

Basic model that uses a 'forest' of decision trees

```
from sklearn.ensemble import RandomForestRegressor

model = RandomForestRegressor(n_estimators=180, max_depth=10, min_samples_leaf=5,
             n_jobs=-1, random_state=18)
model.fit(X, y)
~~~
y_pred =            pd.DataFrame({'sales':model.predict(X_test)})
submission =        pd.concat([test_data['id'], y_pred], axis=1)
submission.to_csv('submission.csv', index=False)
```

BEST SCORE: 1.82088 (V5)

4.3 CatBoostRegressor

Uses decision trees and gradient boosting to form predictions; handles categorical data efficiently

```
from catboost import CatBoostRegressor, Pool

train_pool =        Pool(X_train, y_train, cat_features=cat_features)
val_pool =          Pool(X_val, y_val, cat_features=cat_features)
test_pool =         Pool(X_test, cat_features=cat_features)

model = CatBoostRegressor(loss_function='RMSE', eval_metric='RMSE', depth=12, learning_rate=0.06,
             iterations=3000, od_type='Iter', od_wait=100, task_type='CPU', random_seed=18)
model.fit(train_pool,eval_set=val_pool,verbose=200)
~~~
y_pred =            model.predict(test_pool)
submission =        pd.DataFrame({'id': test_data['id'],'sales': y_pred})
submission.to_csv('submission.csv', index=False)
```

BEST SCORE: 0.60563 (V72)

4.4 LightGBM

Formally known as 'Light Gradient Boosting Machine' is based on a decision trees

```
import lightgbm as lgb

train_set =         lgb.Dataset(X_train, y_train)
val_set =           lgb.Dataset(X_val, y_val)

params =     {'objective': 'regression','metric': 'rmse','boosting_type': 'gbdt',
             'learning_rate': 0.06,'num_leaves': 256,'min_data_in_leaf': 50,
             'lambda_l2': 0.5,'max_bin': 255, 'feature_fraction': 0.8, 'bagging_fraction': 0.8,
             'bagging_freq': 1,'seed': 18}
model = lgb.train(params,train_set,num_boost_round=5000,valid_sets=[train_set, val_set],
             callbacks=[lgb.early_stopping(stopping_rounds=200),lgb.log_evaluation(period=200)])
~~~
y_pred =            model.predict(X_test, num_iteration=model.best_iteration)
submission =        pd.DataFrame({'id': test_data['id'],'sales': y_pred})
submission.to_csv('submission.csv', index=False)
```

BEST SCORE: 1.09858 (V19)

4.5 XGBoost

The "eXtreme Gradient Boosting" model is a highly efficient ensemble machine learning algorithm. Similar to CatBoost and LightGBM, it builds a series of shallow decision trees sequentially, where each new tree specifically corrects the errors and residuals left by the preceding trees. The final prediction is a weighted sum of all the trees' predictions.

```
import xgboost as xgb

dtrain =            xgb.DMatrix(X_train, label=y_train)
dval =              xgb.DMatrix(X_val, label=y_val)
dtest =             xgb.DMatrix(X_test)

params =     {'objective':'reg:squarederror','eval_metric':'rmse','eta': 0.07,
             'max_depth': 14,'subsample': 0.7,'colsample_bytree': 0.7,'lambda': 1.0,'seed': 18}
model = xgb.train(params,dtrain,num_boost_round=5000,evals=[(dtrain, 'train'),
             (dval, 'val')], early_stopping_rounds=200, verbose_eval=200)
~~~
y_pred =            model.predict(dtest, iteration_range=(0, model.best_iteration))
submission =        pd.DataFrame({'id': test_data['id'],'sales': y_pred})
submission.to_csv('submission.csv', index=False)
```

BEST SCORE: 0.84812 (V68)

5 Results

After many versions of the script and different models tried, the best score received from Kaggle (based on RMSLE) was 0.60563 . The metric that was used for training and evaluating the models before submission was RMSE.

5.0.1 Current Metrics : RMSE & RMSLE

RMSE : 'Root mean squared-error' is a standard metric used for evaluating correctness in regression problems. It takes the variance of the prediction to the actual value (i.e. the error), then squares it, takes its average, and finally has its root taken returning it to the original units of the target variable.

The logged version (RMSLE) used by Kaggle is more of a percentage metric showing relative error rather than absolute. It is often used when the target variable has a large range of values or if the data may be skewed.

5.0.2 Analysis of Current Metric

The above two metrics for scoring ultimately show how far off the predictions were from the actual values of the target variable. Since 'sales' of a product category in a specific store during a specific date serves as the target variable and is a continuous numerical variable, this lends to the need of regression models and their companioning metrics showing variance.

There is still opportunity to build on the above workflow and reduce the variance of the predictions towards the actual sales amount that occurs. From completing this work, more accurate predictions of sales can be obtained which can provide insights that a business team can use in decision making.

5.0.3 Proposed Improvements

1 - Adding New Features:

- Further granularity could be achieved by having the product categories dissected further to smaller and more specific groupings. These mini-groups could also serve as value-tags in a new column where a product may be listed in multiple mini-groups simultaneously.
- Certain datasets can be used to find potential features that show collinearity with the target variable; for example a weather related dataset may be able to better indicate days when more beverages are sold.

2 - Exploring datasets of other grocery chains in other countries

- Some may provide already-tuned prediction models that can be tailored to other datasets.
- Trends seen and learned from other sets may drive more local discoveries

6 Conclusion and Looking Forward

There is still room for improvement. Future appendages to this document may include using cross-validation techniques for the hyperparameter tuning, feature importance evaluation, running through all the models again with proper use of one-hot encoding, use of deep learning models, and more.

Appendix A: The Top Scoring Script

Below is the full script used in the Kaggle notebook for the best scored submission to date. It has **RMSLE of 0.60563** and is from Notebook version 72 which uses the CatBoost model :

```
import numpy as np, pandas as pd, matplotlib.pyplot as plt, seaborn as sns

train_data = pd.read_csv(".../train.csv",parse_dates=['date'])
test_data = pd.read_csv(".../test.csv",parse_dates=['date'])
transactions_df = pd.read_csv(".../transactions.csv")
stores_df = pd.read_csv(".../stores.csv")
holidays_df = pd.read_csv(".../holidays_events.csv",parse_dates=['date'])
oil_df = pd.read_csv(".../oil.csv",parse_dates=['date'])
sample_df = pd.read_csv(".../sample_submission.csv")

oil_df['dcoilwtico'] = oil_df['dcoilwtico'].ffill()
oil_df['dcoilwtico'] = oil_df['dcoilwtico'].bfill()

train_data = train_data.merge(stores_df[['store_nbr','city','state']], on='store_nbr', how='left')
test_data  = test_data.merge(stores_df[['store_nbr','city','state']], on='store_nbr', how='left')

train_data = train_data.merge(oil_df[['date','dcoilwtico']], on='date', how='left')
test_data  = test_data.merge(oil_df[['date','dcoilwtico']], on='date', how='left')

hol =               holidays_df.copy()
national =          hol[hol['locale'] == 'National'][['date']]
regional =          hol[hol['locale'] == 'Regional'][['date', 'locale_name']]
local    =          hol[hol['locale'] == 'Local'][['date', 'locale_name']]

regional_index =    (regional.set_index(['locale_name', 'date']).index)
local_index =             (local.set_index(['locale_name', 'date']).index)
train_idx_reg =           pd.MultiIndex.from_frame(train_data[['state', 'date']])
train_idx_loc =           pd.MultiIndex.from_frame(train_data[['city', 'date']])
test_idx_reg =            pd.MultiIndex.from_frame(test_data[['state', 'date']])
test_idx_loc =            pd.MultiIndex.from_frame(test_data[['city', 'date']])

train_data['reg_holi'] =   train_idx_reg.isin(regional_index).astype(int)
test_data['reg_holi']  =   test_idx_reg.isin(regional_index).astype(int)
train_data['loc_holi'] =   train_idx_loc.isin(local_index).astype(int)
test_data['loc_holi']  =   test_idx_loc.isin(local_index).astype(int)

train_data['nat_holi'] =   train_data['date'].isin(national['date']).astype(int)
test_data['nat_holi']  =   test_data['date'].isin(national['date']).astype(int)

train_data['is_holiday'] =       (train_data['nat_holi'] | train_data['reg_holi'] |
train_data['loc_holi']).astype(int)
test_data['is_holiday'] =        (test_data['nat_holi'] | test_data['reg_holi'] |
test_data['loc_holi']).astype(int)

for df in [train_data, test_data]:
    df['time_step'] = (df['date'] - pd.Timestamp('2013-01-01')).dt.days
for df in [train_data, test_data]:
    df["dayofweek"] = df["date"].dt.dayofweek
    df.loc[:, 'is_month_start'] = df['date'].dt.is_month_start.astype(int)
    df.loc[:, 'is_quarter_end'] = df['date'].dt.is_quarter_end.astype(int)
    df.loc[:, 'is_year_end'] = df['date'].dt.is_year_end.astype(int)
full = pd.concat([train_data, test_data], sort=False)

# Sort for correct rolling behavior
full = full.sort_values(['store_nbr', 'family', 'date'])
```

```
# Rolling features (shifted to avoid leakage)
for window in [7, 14, 28]:
    full[f'roll_mean_{window}'] = (full.groupby(['store_nbr', 'family'])['sales']
        .transform(lambda x: x.shift(14).rolling(window).mean()))
# Add lag features (aligned with 14-day forecast horizon)
for lag in [1,7,14, 28, 42]:
    full[f'lag_{lag}'] = (full.groupby(['store_nbr', 'family'])['sales']
        .transform(lambda x: x.shift(lag)))

# Split back
train_data = full[full['sales'].notna()].copy()
test_data  = full[full['sales'].isna()].copy()

train_encoding_base = train_data.copy()
dow_encoding = (train_encoding_base.groupby(['store_nbr', 'family', 'dayofweek'])['sales']
    .mean().reset_index().rename(columns={'sales': 'DOW_mean_sales'}))

train_data = train_data.merge(dow_encoding,on=['store_nbr', 'family', 'dayofweek'],how='left')
test_data = test_data.merge(dow_encoding,on=['store_nbr', 'family', 'dayofweek'],how='left')

from sklearn.preprocessing import LabelEncoder
features = ['time_step', 'store_nbr', 'family','onpromotion','dayofweek','is_holiday',
            'roll_mean_7', 'roll_mean_14', 'roll_mean_28','lag_28','dcoilwtico','is_month_start',
            'is_quarter_end','is_year_end','DOW_mean_sales']

X = train_data.loc[:, features].copy()
y = train_data['sales'].copy()
X_test = test_data.loc[:, features].copy()
cat_features = [features.index('store_nbr'),features.index('family')]
le = LabelEncoder() X[col] = le.fit_transform(X['family']) X_test[col] =
le.transform(X_test['family'])

#Train / Validation Splitting
val_date = pd.Timestamp('2017-07-01')
train_mask = train_data['date']<val_date
val_mask = ~train_mask

X_train, y_train = X[train_mask], y[train_mask]
X_val, y_val = X[val_mask], y[val_mask]
from catboost import CatBoostRegressor, Pool

train_pool =        Pool(X_train, y_train, cat_features=cat_features)
val_pool =          Pool(X_val, y_val, cat_features=cat_features)
test_pool =         Pool(X_test, cat_features=cat_features)

model = CatBoostRegressor(loss_function='RMSE', eval_metric='RMSE', depth=14, learning_rate=0.06,
iterations=3000, od_type='Iter', od_wait=100, task_type='CPU', random_seed=18)
model.fit(train_pool,eval_set=val_pool,verbose=200)

y_pred =                model.predict(test_pool)
submission =        pd.DataFrame({'id': test_data['id'],'sales': y_pred})
submission.to_csv('submission.csv', index=False)
```

Appendix B: Hyperparameter Tuning

This section covers the parameters of the models used; the first block per section shows what was used and the lower block are other parameters that can be used.

B.1.1. RandomForest Regressor

```
n_estimators = 180
        The number of trees in the forest; More trees would reduce variance but increase compute and memory
max_depth = 10
        The max depth of each tree; Limits complexity and overfitting. 'None' would give fully grown trees
min_sample_leaf = 5
        The minimum samples required for a leaf node; Larger values regularize and smooth predictions
n_job = -1
        This value lets all CPU cores be used for parallel training/prediction (speeds up computation)
```

```
max_features = 0.2, 0.33, 0.5, 1, or 'sqrt'
        Features considered per split; Reduce to increase decorrelation between the trees
bootstrap = 'True'
        For sampling with replacement; Use 'False' for deterministic subsampling of full data
min_samples_split = 2 or 0.01
        The minimum samples to split a node; Raise to prevent tiny splits on large datasets
```

B.1.2. CatBoost

```
loss_function = 'RMSE'
        RMSE is the standard objective for regression problems that the model uses to optimize during training
eval_metric = 'RMSE'
        The metric used to evaluate validation performance and for early stopping decisions
depth = 8
        Tree depth which controls the model complexity; larger can fit more complex patterns but risks
        overfitting to the data
learning_rate = 0.07
        The step size for the boosting updates; lower values require more iterations but generalize better
iterations = 3000
        The max number of boosting trees; will stop before reaching it if overfitting detected via the od_type
        and od_wait parameters
od_type = 'Iter' & od_wait = 100
        Overfitting detector type and patience; training stops if no improvement for od_wait iterations
task_type = 'CPU'
        Training done via CPU, not GPU; GPU can be used if you have CUDA-capable hardware.
```

```
l2_leaf_reg = 3 - 10
        Regulation of leaf weights to reduce overfitting; should be done if validation error overfits
bagging_temperature = 0.0 - 1.0
        Controls Bayesian bootstrap randomness for sampling; Use >0 to add randomness instead of subsample
subsample = 0.6 - 0.9
        Fraction of data used per tree (Bernoulli sampling); alternative to bagging_temperature
grow_policy = 'Depthwise' or 'Lossguide'
        Lossguide is used for large datasets or when may leaves are desired
min_data_in_leaf = 1 - 20
        Minimum samples required in a leaf; Increase it to reduce overfitting on noisy data
random_strength = 1 - 20
        Adds randomness to split scoring, avoiding overfitting; increase slightly if model overfits
```

B.1.3. LightGBM

```
objective : 'regression' & metric = 'rmse'
        These are the standard choices for regression problems
boosting_type : 'gbdt'
        Gradient-boosted decision trees
learning_rate : 0.06
        A lower value would improve generalization but require more rounds/iterations
num_leaves : 256
        The main complexity knob for leaf-wise trees; larger would be more expressive but risk overfitting
min_data_in_leaf : 50
        The minimum samples per leaf to prevent tiny overfitting leaves; Needs raising for noisy/small datasets
lambda_l2 : 0.5
        L2 regularization on leaf weights to reduce overfitting
max_bin : 255
        Number of histogram bins for continuous features ; larger values capture finer splits but increase
        computation time
feature_fraction : 0.8
        Fraction of features sampled per iteration (feature bagging) to decorrelate trees
bagging_fraction : 0.8 & bagging_freq : 1
        Row subsampling fraction and frequency to add randomness
```

```
max_depth : 8 - 16
        Gives a hard limit on tree depth to control overfitting
min_split_gain : 0.0 - 0.1
        The minimum gain to make a split to regularize small gains
```

B.1.4. XGBoost

```
objective : 'reg:squarederror' & eval_metric : 'rmse'
        Standard choices for regression problems
eta : 0.07
        The learning rate that scales each tree's contribution; lower values generalize better but are slower
max_depth : 14
        The max tree depth; controls complexity and affects bias/variance
subsample : 0.7 & colsample_bytree : 0.7
        The row and column sampling fractions per tree to add stochasticity and reduce overfitting
lambda : 1.0
        L2 regularization on leaf weights; increases model smoothness and reduces overfitting
```

```
min_child_weight : 1 - 10
        The minimum sum of instance weight in a child; avoid splitting on small/noisy partitions by increasing
gamma : 0 - 2
        The minimum loss reduction to make a split; higher values give more conservative splits
reg_alpha : 0 - 1
        L1 regularization on leaf weights; used to induce sparsity and reduce overfitting
tree_method : 'hist' or 'gpu_hist'
        'gpu_hist' should be used if GPU is available for computation speed
```

Appendix C: Codes for Plots

Below are the blocks of code used to create the graphs seen in the initial EDA.

```
# Initial line to ensure high-resolution outputs for the graphs
%config InlineBackend.figure_format = 'retina'
```

```
# Graph 2.4.1-A - Total Sales over time for all stores and all product families
agg = train_data.groupby("date")["sales"].sum()
roll = agg.rolling(7).mean().shift(14)

plt.figure(figsize=(14, 5))
sns.lineplot(x=agg.index, y=agg.values, label="Daily Sales")
sns.lineplot(x=roll.index, y=roll.values, color="red", linewidth=2,
             label="7-Day Rolling Mean (Shifted 14 Days)")

plt.title("Total Sales Over Time (All Stores, All Families)")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.grid(True)
plt.legend()
plt.show()

# Graph 2.4.1-B - Total Sales over time for three specific stores
stores = [1, 25, 50]

fig, axes = plt.subplots(3, 1, figsize=(14, 12), sharex=True)

for ax, store in zip(axes, stores):
    temp = train_data[train_data["store_nbr"] == store].groupby("date")["sales"].sum()
    roll = temp.rolling(7).mean().shift(14)
    sns.lineplot(x=temp.index, y=temp.values, ax=ax, label="Sales")
    sns.lineplot(x=roll.index, y=roll.values, ax=ax,
                 color="red", linewidth=2,
                 label="7-Day Rolling Mean (Shifted)")

    ax.set_title(f"Total Sales Over Time - Store {store}")
    ax.set_ylabel("Sales")
    ax.grid(True)
    ax.legend()

plt.xlabel("Date")
plt.show()

# Graph 2.4.1-C - Total Sales of specific family of product at the three stores
family = "BEVERAGES"
stores = [1, 25, 50]

fig, axes = plt.subplots(3, 1, figsize=(14, 12), sharex=True)

for ax, store in zip(axes, stores):
    temp = train_data[(train_data["store_nbr"] == store) & (train_data["family"] == family)]
    temp = temp.groupby("date")["sales"].sum()

    roll = temp.rolling(7).mean().shift(14)

    sns.lineplot(x=temp.index, y=temp.values, ax=ax, label="Sales")
    sns.lineplot(x=roll.index, y=roll.values, ax=ax,
                 color="red", linewidth=2,
                 label="7-Day Rolling Mean (Shifted)")

    ax.set_title(f"{family} Sales Over Time - Store {store}")
```

```
    ax.set_ylabel("Sales")
    ax.grid(True)
    ax.legend()
plt.xlabel("Date")
plt.show()
```

Graph 2.4.2-A - HeatMap showing correlation of numeric variables (including Target Variable)

```
agg = train_data.groupby("date").agg({
    "sales": "sum",
    "onpromotion": "sum",
    "dcoilwtico": "mean",
    "lag_28": "mean",
    "dayofweek": "mean"
}).rename(columns={"sales": "sales_AS_AF","onpromotion": "onpromotion_AS_AF","lag_28":
"lag_28_AS_AF"})

agg["dayofweek"] = agg["dayofweek"].round().astype(int)

corr = agg.corr()

plt.figure(figsize=(10, 7))
sns.heatmap(corr,annot=True,cmap="coolwarm",fmt=".2f",linewidths=0.5,square=True)
plt.title("Correlation Heatmap – Aggregated All-Store All-Family Features")
plt.show()
```